블록체인(01)

블록체인(01) — 중간고사 연습문제 100제 (2): 8주차 솔리디티 포함

범위: 0. Introduction ~ 9. 솔리디티(8주차)까지. 강의자료 표현과 슬라이드 예제 코드를 기준으로 출제했습니다.
풀이 칸에 직접 풀고, 문항 오른쪽 □에 채점 표시하세요. ⭐ = 출제 가능성이 특히 높은 문제 · 정답과 해설은 맨 뒤 **정답 및 해설**에 있습니다.

날짜 _____ 점수 _____ / 100 틀린 문항 _____

1. 블록체인 탄생 (1~8)

1. (O/X) 서브프라임 모기지에서 '서브프라임'은 가장 높은 신용 등급을 뜻하며, '모기지'는 부동산을 담보로 자금을 대출해 주는 제도이다. □

2. (단답) 강의에서 제시한 **화폐의 요건** 네 가지를 쓰시오. □

3. (객관식) **금화본위제**가 금핵본위제로 전환된 이유와 거리가 **먼** 것은? □
① 금화를 들고 다니기 불편했다
② 사용 과정에서 화폐(금화)가 훼손되었다
③ 금화가 가치 저장 수단으로만 쓰이고 잘 유통되지 않았다
④ 금화는 금 보유량과 무관하게 무제한 발행할 수 있었다

4. ⭐ (단답) 신용화폐는 금과 같은 담보 없이 무엇에 의존해 가치가 부여되는가? 또한 신용화폐의 무분별한 발행으로 하이퍼인플레이션을 겪은 국가를 두 곳 이상 쓰시오. □

5. (단답) 2013년 에드워드 스노든이 폭로한, 미국 국가안보국(NSA)의 개인정보 무차별 수집·사찰 프로그램의 이름은? □

6. ⭐ (매칭) 블록체인 관련 암호 기술과 블록체인에서의 쓰임을 연결하시오. □
기술: (A) 공개키 암호(서명) (B) 해시 함수(SHA-3 등) (C) 머클 트리 (D) 영지식 증명 (E) 멀티 서명
쓰임: ㉠ 트랜잭션을 효율적으로 묶어 검증 ㉡ 지갑 및 주소 생성 ㉢ 정보를 숨기며 유효성만 증명 ㉣ 블록 연결, 합의 알고리즘 ㉤ 지갑 안전성 제공

7. (서술) 데이비드 차움이 1989년 설립한 DigiCash가 기술적 장점에도 불구하고 실패(1998년 파산)한 이유를 쓰시오. □

8. (빈칸) 비트코인 백서: "화폐 거래의 신뢰를 보장하기 위해 그동안 (①)에 전적으로 의존해 왔지만, 이제 그 대신 (②)에 기반하여 개인 간 직접 거래가 가능한 전자적 화폐 시스템이 필요하다. 금 채굴에 노력이 드는 것처럼 비트코인에서는 (③)과 (④)라는 자원을 통해 화폐를 발행한다." □

2. 블록체인 기초 (9~17)

9. ⭐ (매칭) 블록체인 등장 배경(필요)과 블록체인 설계 요소를 연결하시오. □
필요: (A) 중앙 시스템·독점 운영 탈피 (B) 투명성 확보 (C) 화폐 발행·공급 통제
설계: ㉠ 분산 장부 ㉡ 금본위제 기반 설계 ㉢ 탈중앙화 시스템

10. (단답) 트랜잭션이 일반적으로 기록하는 세 요소(누가, 무엇을, 어떻게)에 해당하는 항목 이름 세 가지를 쓰고, 트랜잭션의 예를 두 가지 쓰시오. □

11. (O/X) 블록 해시는 블록을 대표하는 식별자이지만, 실제 블록 데이터 안에는 포함되지 않기도 한다. □

12. ⭐ (시나리오) eCash 방식에서 사용자가 이미 사용한 화폐(Id: 2334)를 다시 제출했다. 발행 기관의 처리 과정과 결과를 쓰시오. □

13. ⭐ (표 완성) 금본위제와 블록체인(비트코인)을 비교하시오. □

항목	금본위제	블록체인(비트코인)
채굴	금 채굴	(①)
희소성	한정된 매장량	(②)
발행량 예측	보관된 금 범위에서만 발행	(③)

14. (빈칸) 블록체인 분류에서 **기능 관점**은 "누가 검증·블록 생성 권한을 가지는가"를 기준으로 (①)/(②) 블록체인으로 나누고, **운영 관점**은 "네트워크를 누가 관리·운영하는가"를 기준으로 (③)과 Private/Hybrid/Consortium으로 나눈다. □

15. (객관식) 하이브리드 블록체인에 대한 설명으로 옳은 것은? □
① 단일 조직이 운영하며 승인받은 내부 사용자만 이용한다
② 개방형과 폐쇄형을 병행 운영하여 공공성과 프라이버시의 균형을 추구하지만 설계가 복잡하다
③ 여러 기관이 공동 운영하며 권한이 기관 간에 분산된다
④ 누구에게나 허가 없이 모든 권한을 부여하는 블록체인의 원형이다

16. (매칭) 블록체인 응용 사례와 분야를 연결하시오. □
사례: (A) 원산지부터 소비자까지 이동 경로 추적 (B) 탈중앙화 자율조직(DAO) (C) 실물 자산을 토큰화해 등기·소유권 이전을 스마트 계약으로 진행 (D) 조세·복지 서비스 기록 관리
분야: ㉠ 투표 및 거버넌스 ㉡ 공공 서비스 ㉢ 기록 관리 ㉣ P2P 거래

17. ⭐ (순서 배열) DID(탈중앙화 신원)의 동작 흐름을 순서대로 배열하시오. □
(가) 사용자가 필요한 정보만 공개 (나) 블록체인에 DID 발급 (다) 블록체인을 통해 유효성 검증 (라) 서비스 제공자의 신원 확인 요청 (마) 개인 지갑(앱)에 신원 정보 저장

3. 블록체인을 위한 암호기술 (18~29)

18. (단답) 해시 함수 SHA-1이 생성하는 해시 값의 길이(비트)는? 또한 SHA-1이 현재 권장되지 않는 이유를 쓰시오. □

19. ⭐ (매칭) 강의의 '지문' 비유와 해시 함수 보안 성질을 연결하시오. □
비유: (A) 지문을 보고 누구인지 알 수 있는가? (B) Alice와 같은 지문을 가진 사람이 누구인지 알 수 있는가? (C) 지문이 같은 서로 다른 두 사람을 찾을 수 있는가?
성질: ㉠ 충돌 저항성 ㉡ 역상 저항성 ㉢ 제2역상 저항성

20. (서술) 소프트웨어 배포 사이트가 파일과 함께 해시 값을 제공하는 이유를 설명하시오. □

21. (매칭) 해시 함수와 특징을 연결하시오. □
함수: (A) MD5 (B) SHA-2 (C) BLAKE2 (D) Poseidon
특징: ㉠ 224/256/384/512비트 출력, 현재까지 사용되지만 잠재적 취약점 존재 ㉡ ChaCha 기반·병렬 최적화로 매우 빠름, RFC 7693 ㉢ 과거 널리 쓰였으나 취약점이 발견되어 권장되지 않음 ㉣ 대수적 연산 중심으로 영지식 증명에 유리

22. (빈칸) SHA-3 공모전은 2008년 1차 라운드 (①)개 후보 → 2009년 2차 (②)개 → 2010년 3차 (③)개 최종 후보를 거쳐 2012년 10월 (④)가 선정되었다. □

23. ⭐ (단답) 트랜잭션 Tx1~Tx4로 머클 트리를 만들었다(해시1~해시4 → 해시12, 해시34 → 루트해시). **Tx3**의 머클 증명을 구성하는 값을 모두 쓰시오. □

24. (계산) 트랜잭션이 1,024개인 머클 트리에서 특정 트랜잭션의 포함 여부를 증명할 때 필요한 형제 해시 값은 몇 개인가? □

25. ⭐ (서술) 서명 s = Sig_sk(h(m))을 받은 검증자가 서명을 검증하는 절차를 쓰시오. □

26. (서술) RSA 서명 σ = m^d mod n에서 d와 n이 각각 무엇인지 쓰고, RSA가 블록체인에 적합하지 않은 이유를 쓰시오. □

27. (O/X) EdDSA는 서명할 때마다 새로운 난수를 생성하므로 연산 과정이 일정하지 않아 부채널 공격에 취약하다. □

28. ⭐ (시나리오) 2-of-3 멀티 서명 지갑이 있다. ① 키 1개가 유출되었을 때 공격자가 유효한 서명을 만들 수 있는가? ② 키 1개를 분실했을 때 정상 사용자는 여전히 서명할 수 있는가? □

29. (계산·응용) 영지식 증명 동굴 예시에서 주문을 **모르는** 증명자가 한 번의 시행을 우연히 통과할 확률은 1/2이다. 10번 연속 통과할 확률은? 이 확률이 작다는 것과 관련된 영지식 증명의 성질은? □

4. 합의 알고리즘 (30~41)

30. (순서 배열) 정직한 노드만 있다고 가정할 때 블록 생성 및 기록 과정을 배열하시오. □
(가) 대표블록 동기화 (나) 후보블록 생성 (다) 대표블록 선정

31. ⭐ (매칭) 합의 알고리즘이 대응해야 하는 위협과 설명을 연결하시오. □
위협: (A) 패킷 지연·손실 (B) 검열 & 블로킹 (C) 스팸 (D) 이중 지불
설명: ㉠ 특정 트랜잭션을 의도적으로 제외 ㉡ 같은 시각에 서로 다른 블록을 생성하거나 받게 됨 ㉢ 서로 다른 상대에게 동일한 자산을 보내는 트랜잭션 전파 ㉣ 불필요한 메시지를 대량 전송해 자원 소모·서비스 장애 유발

32. (단답) 체인 분기(Chain Fork)가 발생하는 원인을 네 가지 쓰시오. □

33. ⭐ (O/X) 최장 체인 규칙(Longest Chain Rule)은 시간이 지날수록 한 가지 체인으로 수렴하므로 **결정적(즉시) 최종성**을 제공한다. □

34. (시나리오) 블록 A 뒤에 블록 B와 C가 동시에 생성·전파되었다. 이후 어떤 채굴자가 블록 B 뒤에 블록 D를 연결해 전파했다. 최장 체인 규칙에 따라 최종적으로 선택되는 체인과, 블록 C를 부르는 이름을 쓰시오. □

35. (O/X) SegWit은 기존 노드와 호환되지 않는 규칙 변경이었기 때문에 비트코인과 비트코인 캐시로 체인이 나뉜 하드포크이다. □

36. ⭐ (순서 배열) PoW 채굴자의 동작 과정을 배열하시오. □
(가) 퍼즐 풀기(반복 시행) (나) 대표 블록 선정 시 보상 (다) 트랜잭션 묶음 후보 구성 (라) 성공 시 블록에 증명을 담아 전파 (마) 퍼즐 입력 및 난이도 확인

37. (단답) 비트코인 채굴에 등장한, 특정 용도 전용 채굴 장비의 영문 약어는? 이것이 PoW 퍼즐 설계에서 시사하는 점은? □

38. (단답) PoS에서 검증 과정 중 부정행위를 한 검증자의 지분을 삭감하는 것을 무엇이라 하는가? □

39. ⭐ (서술) DPoS에서 블록이 승인되었을 때 보상을 받는 대상은 누구인가? DPoS의 장점과 단점을 각각 쓰시오. □

40. ⭐ (서술) 비잔틴 장군 문제 예시(4개 부대 중 3개 이상이 동시에 공격해야 승리, C가 배신자)에서 ① 메시지를 시계 방향으로 전달할 때와 ② 메시지 전송 방법을 바꿔 2/3 다수결로 판단할 때의 결과를 각각 쓰시오. □

41. ⭐ (객관식) 합의 알고리즘 비교로 **옳지 않은** 것은? □

① PoW의 분기 선택은 유효한 체인 중 누적 작업량이 최대인 체인이다
② PoS(이더리움)는 LMD-GHOST로 지분 가중 투표를 반영해 분기를 선택한다
③ PBFT는 참여자가 늘수록 통신량이 줄어 대규모 개방형 네트워크에 적합하다
④ DPoS는 적은 합의 참여자로 통신 부담이 감소하지만 대표 집중·담합 위험이 있다

5. 비트코인 (42~55)

42. ⭐ (단답) 강의에서 정리한 'Satoshi's Solution(2008)'의 네 가지 구성 요소를 쓰시오. □

43. (서술) "주소"의 개념이 비트코인과 이더리움에서 어떻게 다른지 쓰시오. □

44. (매칭) 비트코인 노드의 기능 모듈과 역할을 연결하시오. □
모듈: (A) Wallet (B) Full Blockchain (C) Network Routing Node (D) Miner
역할: ㉠ P2P 통신, 블록 전달, 피어 연결 ㉡ 작업 증명으로 블록 생성·전파 ㉢ 키 관리, 주소 생성, 트랜잭션 발송, 잔액 조회 ㉣ 전체 블록체인 저장, 블록 동기화, 트랜잭션 검증

45. ⭐ (객관식) SPV(Lightweight wallet) 노드에 대한 설명으로 **옳지 않은** 것은? □
① 블록체인 전체가 아니라 블록 헤더 정보만 가진다
② 새 블록이 생성되면 풀 노드에게서 헤더를 받아 헤더 체인에 연결한다
③ 트랜잭션 포함 여부를 확인하려면 풀 노드에게 관련 정보를 요청한다
④ 모든 기능을 다 갖춘 초창기 표준 노드로, Bitcoin Core라고도 한다

46. (단답) 노드가 새로 받은 트랜잭션을 검증할 때 확인하는 세 가지 항목과, 검증을 통과한 트랜잭션이 저장되는 곳의 이름을 쓰시오. □

47. ⭐ (단답) 노드가 새로 받은 **블록**을 검증할 때 확인하는 네 가지 항목을 쓰시오. □

48. (빈칸) P2PKH에서 UTXO는 (①)의 해시 값으로 잠겨 있으며, 그에 대응하는 (②)를 가진 사람만 서명을 만들어 잠금을 해제할 수 있다. 다중 서명처럼 복합 조건을 쓰는 잠금 방식은 (③)이다. □

49. ⭐ (계산) A의 지갑에는 2 BTC, 1.5 BTC, 0.7 BTC짜리 UTXO가 있다. A가 B에게 3 BTC를 보내고 수수료로 0.05 BTC를 지불하려 한다. 2 BTC와 1.5 BTC UTXO를 입력으로 쓴다면 출력은 어떻게 구성되는가? 거래 후 A의 잔액은? □

50. (계산) 어떤 블록의 블록 보조금이 3.125 BTC이고, 블록 안 트랜잭션 수수료 총합이 0.42 BTC이다. 코인베이스 트랜잭션의 출력 금액은? □

51. ⭐ (계산) 최초 블록 보조금이 50 BTC일 때, 블록 높이 450,000에서의 블록 보조금은? □

52. ⭐ (계산) nBits = 0x17034219일 때 ① E(10진수)와 C를 구하고 ② 32바이트 타겟 T를 16진수로 썼을 때 C 앞의 0 바이트 수와 C 뒤의 0 바이트 수를 각각 구하시오. □

53. (O/X) nBits = 0x1d80ffff는 유효한 난이도 값이다. □

54. (계산) 비트코인 난이도를 다시 계산하는 주기인 2,016블록이 정확히 평균 생성 시간대로 만들어졌다면 걸리는 시간은 며칠인가? □

55. (O/X 묶음) 각 문장의 O/X를 판단하시오. □

(a) 트랜잭션 카운터는 블록 해시 계산에 포함된다.

(b) 블록 탐색기의 원시 블록 데이터는 리틀엔디안으로 읽어야 한다.

(c) 대표 블록에 포함된 트랜잭션은 메모리풀에서 삭제된다.

(d) 채굴자는 수수료와 관계없이 도착한 순서대로 메모리풀의 트랜잭션을 블록에 담아야 한다.

6. 이더리움 기초 (56~64)

56. (단답) ① 이더리움 최초 작동일 ② 이더리움이 PoS로 전환된 날짜를 쓰시오. □

57. (O/X) 2021년 #666,833 블록 근처에서 보도된 RBF 이중 지불 사건은 실제 공격으로 확인되었다. □

58. (단답) PoW 51% 공격 위험 사례로, ① 2014년 전체 해시레이트의 50%에 육박했던 마이닝 풀과 ② 2025년 합산 해시 파워가 한때 50%를 넘긴 두 마이닝 풀을 쓰시오. □

59. ⭐ (객관식) 이더리움 계정 모델의 장점으로 강의에서 제시한 것이 **아닌** 것은? □

① 이중 지불 문제 해결이 쉽다
② 'balance'로 잔액을 효율적으로 업데이트한다
③ 모든 거래가 이전 출력을 참조하므로 개인정보 추적이 어렵다
④ 프로그래밍 시 계정 간 잔액 조회 및 송금이 편리하다

60. (서술) 스마트 컨트랙트를 자판기에 비유해 설명하고, '결정적(deterministic)으로 반응한다'는 말의 의미를 쓰시오. □

61. ⭐ (순서 배열) 스마트 컨트랙트의 작성·배포 과정을 배열하시오. □
(가) 블록에 포함된 후 계약 계정 생성 (나) 컨트랙트 코드 작성 (다) 해당 주소에 컨트랙트 코드 저장(이후 불변) (라) 배포 트랜잭션 생성 및 전송

62. ⭐ (표 완성) 이더리움과 비트코인을 비교하시오. □

항목	이더리움	비트코인
스마트 컨트랙트	지원((①))	제한적(튜링-불완전)
트랜잭션 기록 모델	(②)	(③)
현재 블록 생성 시간	~(④)초	평균 약 (⑤)분
블록 용량 제한	(⑥) 기반	현재 최대 4M Weight

63. ⭐ (계산) gas limit 100,000, gas price 15 gwei인 트랜잭션이 60,000 gas를 사용하고 정상 종료되었다. ① 실제 수수료(gwei, ETH) ② 환불되는 금액(gwei)을 구하시오. 같은 트랜잭션이 무한 루프로 실패했다면 수수료는? □

64. (O/X) 이더리움의 수수료 팁(Priority Fee)은 검증자가 수취하며, 팁이 높을수록 블록에 선택될 가능성이 높아진다는 점에서 비트코인의 거래 수수료와 같은 개념이다. □

7. 이더리움 (65~75)

65. ⭐ (서술) CA(계약 계정)가 스스로 새로운 트랜잭션을 발생시킬 수 없는 이유와, CA가 동작하는 방식을 쓰시오. □

66. (단답) 계정 구조의 `nonce` 필드가 ① EOA와 ② CA에서 각각 무엇을 나타내는지 쓰시오. □

67. (O/X) `codeHash`는 EVM 바이트코드의 keccak256 해시로, 다른 필드와 마찬가지로 컨트랙트 실행에 따라 갱신될 수 있다. EOA는 이 필드에 공백 문자의 해시를 저장한다. □

68. (계산) ① 0.25 ETH는 몇 Wei인가? ② 1 gwei는 몇 Wei인가? □

69. ⭐ (계산·단답) EOA 주소는 공개키 해시의 마지막 20바이트를 사용한다. ① 이는 몇 비트인가? ② 16진수로 쓰면 '0x'를 제외하고 몇 글자인가? ③ 추가 문자열 없이 대소문자만으로 주소 오타를 검출하는 표준은? □

70. (객관식) 이더리움 노드에 대한 설명으로 옳은 것은? □
① 풀 노드는 모든 과거 상태를 영구히 보관한다
② 라이트 노드는 블록과 상태를 모두 직접 검증한다
③ 아카이브 노드는 과거 시점 조회를 위해 모든 과거 상태를 보관하며 2025년 기준 12-14TB가 필요하다
④ 아카이브 노드는 블록 헤더만 저장한다

71. ⭐ (매칭) PoS 전환 이후 이더리움 블록 정보 필드와 설명을 연결하시오. □
필드: (A) Proposed On (B) Withdrawals (C) Fee Recipient (D) Burnt Fees (E) Nonce
설명: ㉠ 블록 보상 수령자 주소 ㉡ 소각된 ETH 총량(Base fee) ㉢ PoS 전환 이후 0 ㉣ 검증자가 제안한 슬롯/에포크 정보 ㉤ 스테이킹 보상 인출 기록

72. ⭐ (계산) EIP-1559 트랜잭션: Max Fee 30 gwei, Base Fee 20 gwei, Priority Fee 3 gwei, 사용 가스 50,000. ① 총 수수료 ② 소각량 ③ 검증자 수취액 ④ Max Fee 기준 대비 돌려받는 금액을 gwei로 구하시오. □

73. (계산·단답) Legacy 트랜잭션의 서명 V, R, S의 크기를 각각 쓰고 합계 바이트를 구하시오. 또한 CA를 생성하는 트랜잭션에서 `value` 필드는 어떤 의미가 되는가? □

74. ⭐ (계산·빈칸) ① 에포크 1개(32슬롯)는 몇 초인가? ② 분기 시 메인 시퀀스를 정하는 프로토콜은 (㉠), 블록에 최종성을 보장하는 프로토콜은 (㉡)이며 스테이킹 지분의 (㉢) 투표로 확정한다. □

75. (순서 배열) 이더리움이 트릴레마 해결을 위해 진행한 업데이트를 시간순으로 배열하시오. □
(가) 블롭 용량 확대 (나) PoS 전환 (다) 데이터 샤딩/블롭 도입 (라) L2 솔루션 활성

8. DApp 개발환경 구축 (76~83)

76. ⭐ (분류) 다음 도구를 강의의 네 가지 분류에 맞게 나누시오. □

도구: Remix IDE, Hardhat, Geth, Infura, Ganache, Sepolia, MetaMask, ethers.js

분류: (A) 스마트 계약 개발 도구 (B) 노드 기능 제공 도구 (C) 테스트 네트워크 도구 (D) 지갑 및 사용자 인터페이스 도구

77. (서술) Remix IDE와 Hardhat의 공통점과, Hardhat이 특히 유리한 상황을 쓰시오. □

78. (O/X) Ganache는 로컬에서 빠르게 실행되는 테스트용 블록체인으로, 현재도 활발하게 업데이트가 지원된다. □

79. (단답) Infura·Alchemy 같은 서비스는 무엇을 제공하며, 이를 쓰면 어떤 점이 편리한가? □

80. ⭐ (O/X) DApp을 만들려면 Node.js 서버와 DB 구축이 반드시 필요하다. □

81. (매칭) Remix 좌측 패널 기능과 설명을 연결하시오. □

기능: (A) 컴파일 (B) 배포 & 실행 (C) 디버거 (D) 플러그인 관리자

설명: ㉠ 트랜잭션 실행 흐름을 단계별로 추적 ㉡ 네트워크 선택, 계약 배포, 함수 호출, 트랜잭션 전송 ㉢ 추가 도구 설치 및 관리 ㉣ .sol 컴파일, 자동 컴파일 설정, 오류/경고 확인

82. (단답) ① Remix 배포 화면의 Value 입력 단위는? ② Remix를 Sepolia에 연결하려면 Environment 드롭다운에서 어떤 항목을 선택해야 하는가? □

83. ⭐ (O/X 묶음) □

(a) MetaMask의 Sepolia 주소는 실제 이더리움 메인넷 주소와 동일하다.

(b) Sepolia에서는 실제 ETH 대신 테스트용 토큰(SepoliaETH)을 쓰므로 계약 배포·실행에 가스 수수료가 발생하지 않는다.

(c) `public`으로 선언한 상태 변수는 같은 이름의 getter 함수가 자동 생성된다.

(d) 비밀복구구문은 다른 환경에서 지갑을 복구할 때 쓰이므로 반드시 별도로 저장해야 한다.

9. 솔리디티 — 8주차 (84~100)

84. (O/X) 솔리디티는 동적 타입 언어여서 변수의 타입을 실행 시점에 검사하며, 구문은 Python에 가깝다. □

85. ⭐ (서술) '솔리디티 모노컬처'의 의미를 쓰고, 그 장점과 단점을 각각 쓰시오. □

86. ⭐ (매칭) 라이선스와 설명을 연결하시오. □
라이선스: (A) MIT (B) Apache-2.0 (C) GPL-3.0 (D) Unlicense
설명: ㉠ 수정·배포를 허용하지만 결과물도 같은 라이선스로 배포하고 코드 공개 필수 ㉡ 저작권을 최대한 포기하고 누구나 자유롭게 사용 ㉢ 저작권 고지만 하면 상업적 사용까지 허용 ㉣ ㉢과 비슷하지만 특허 관련 조항이 명시됨

87. ⭐ (판별) 각 pragma 지시문에 대해 컴파일러 버전 0.8.2, 0.8.3, 0.8.25, 0.9.0 중 **호환되는 것을 모두** 고르시오. □
(a) `pragma solidity ^0.8.3;`
(b) `pragma solidity >0.8.3;`
(c) `pragma solidity 0.8.3;`
(d) `pragma solidity 0.8.2 || 0.8.3;`

88. ⭐ (코드 해석) 다음 코드에 대해 답하시오. □

```
// SPDX-License-Identifier: MIT
pragma solidity 0.6.8;
import "./MyLib.sol";

contract HelloWorld {
    string private greet;

    constructor(){
        greet = "Hello World";
    }

    function getString() public view returns(string memory){
        return greet;
    }
}
```

① 이 코드가 정상 동작하는 컴파일러 버전은? ② 첫 줄을 생략하면 어떻게 되는가? ③ `constructor`는 언제, 몇 번 실행되는가? ④ `greet`를 외부에서 읽으려면 왜 `getString()`이 필요한가?

89. ⭐ (매칭) EVM의 데이터 저장 영역과 설명을 연결하시오. □

영역: (A) 스토리지 (B) 메모리 (C) 호출데이터(calldata) (D) 스택

설명: ㉠ 함수 실행 중에만 존재하는 휘발성 임시 공간 ㉡ 컨트랙트 상태 변수가 영구히 저장되는 공간 ㉢ EVM 내부 계산용 임시 공간, 1024단계 이상이면 예외 ㉣ 외부에서 함수를 호출할 때 입력값이 저장되는 읽기 전용 임시 공간

90. ⭐ (시나리오) 사용자 Alice(EOA)가 컨트랙트 A의 함수를 호출했고, 그 함수 안에서 컨트랙트 B의 함수를 호출했다. **컨트랙트 B 안에서** `msg.sender`와 `tx.origin`의 값은 각각 누구의 주소인가? □

91. (표 완성) 상태 변수와 지역 변수를 비교하시오. □

항목	상태 변수	지역 변수
선언 위치	(①)	함수 내부
저장 위치	컨트랙트 고유 스토리지 (CA의 (②)로 식별)	(③)
수명	블록체인에 영구 저장, 다음 트랜잭션에도 반영	(④)
가스	읽기·쓰기 모두 소모	상대적으로 매우 (⑤)

92. (코드 해석) 다음 코드의 `str`, `num1`, `num2`, `addr`, `block.timestamp`, `msg.sender`를 상태 변수 / 지역 변수 / 전역 변수로 분류하시오. □

```
pragma solidity >=0.8.0 <0.9.0;

contract HelloWorld {
    string public str = "Hello World";

    function getInfo() public view returns(uint, uint, address){
        uint num1 = 2;
        uint num2 = block.timestamp;
        address addr = msg.sender;
        return (num1, num2, addr);
    }
}
```

93. ⭐ (분류) 다음 자료형을 값 타입과 참조 타입으로 나누시오. □

`bool`, `uint`, `address`, `string`, 배열, `struct`, `mapping`, `enum`, `bytes32`, `bytes`

94. ⭐ (표 완성) 참조 타입 데이터를 대입할 때 복사되는지(사본 생성), 참조되는지(같은 공간 공유) 쓰시오. □

소스 → 타겟	결과
memory → memory	(①)
memory → storage	(②)
storage → memory	(③)
storage → storage	(④)
calldata → memory	(⑤)

95. (계산·단답) ① `int8` 과 `uint8` 이 표현할 수 있는 값의 범위를 10진수로 쓰시오. ② `uint` 는 어떤 타입의 별칭인가? ③ 솔리디티에서 지수 연산자는? □

96. ⭐ (코드 해석) Remix에서 각 함수를 호출했을 때의 반환값을 쓰시오. □

```
contract Integers{
    function getTwo() public pure returns(uint){
        uint a = 3;
        uint b = 2;
        return a / b * 2;
    }

    function getThree() public pure returns(uint){
        return 3 / 2 * 2;
    }

    function shift() public pure returns(uint[2] memory){
        uint[2] memory a;
        a[0] = 16 << 4;
        a[1] = 16 >> 3;
        return a;
    }
}
```

97. ⭐ (객관식) 주소 타입의 멤버에 대한 설명으로 **옳지 않은** 것은? □

① `balance`는 계정 주소에 있는 이더를 wei 단위로 반환한다
② `transfer`는 송금에 실패하면 트랜잭션이 되돌려진다
③ `send`는 송금에 실패해도 트랜잭션을 되돌리지 않고 `false`를 반환한다
④ `delegatecall`은 다른 컨트랙트의 코드와 저장소를 모두 그 컨트랙트 것으로 사용한다
⑤ `transfer`, `send`, 송금 목적의 `call`은 `address payable` 타입에서만 사용할 수 있다

98. (코드 해석) 다음에서 ① `b[0]` 과 `b[1]` 에 저장되는 값은? ② `fixedBytesArray` 는 값 타입을, `dynamicBytesArray` 는 참조 타입을 반환한다. 그 이유를 반환 타입 선언에서 찾아 설명하시오. □

```
contract Bytes{
    function fixedBytesArray() public pure returns (bytes2){
        bytes2 b = "ba";
        return b;
    }

    function dynamicBytesArray() public pure returns (bytes memory){
        bytes memory a = 'baaaaaaaaaa';
        return a;
    }
}
```

99. ⭐ (코드 해석) 배포 직후 ① `getColor()` ② `getDefaultcolor()`를 호출한 결과와, ③ `setColor()`를 호출한 뒤 `getColor()`의 결과를 쓰시오. □

```
contract Enum {
    enum Colors {Red, Blue, Green}

    Colors color;
    Colors constant defaultColor = Colors.Green;

    function setColor() public {
        color = Colors.Blue;
    }

    function getColor() public view returns (Colors){
        return color;
    }

    function getDefaultcolor() public pure returns(uint){
        return uint(defaultColor);
    }
}
```

100. ⭐ (코드 해석) 다음 코드에 대해 답하시오. □

```
contract Selector{
    function e() external pure returns(bytes4){
        return this.e.selector;
    }

    function f() public pure returns(uint){
        return g();
    }

    function g() internal pure returns(uint){
        return 100;
    }
}
```

① `f()` 의 반환값은? ② `this.e` 와 `g()` 는 각각 내부/외부 함수 타입 중 무엇인가? ③ `e()` 가 반환하는 `selector` 는 무엇이며 몇 바이트인가? ④ 외부 계정이 Remix에서 `g()` 를 직접 호출할 수 있는가?

정답 및 해설

1. 블록체인 탄생 (1~8)

1. **X** — 서브프라임은 일정 기준 이하의 저소득층 등급, 즉 **가장 낮은** 등급입니다. 모기지 설명은 맞습니다.
2. **내재적 가치 / 희소성 / 사용성(편리성·휴대성·보관 용이성) / 보편성**
3. **④** — 금본위제에서는 금 보유량 이상으로 발행할 수 없습니다. ①②③은 금화본위제의 한계입니다.
4. **정부의 법적 보증과 중앙은행의 신뢰**(발행자의 신용)에 의존합니다. 예: **아르헨티나, 베네수엘라, 짐바브웨**
5. **프리즘(PRISM)**
6. **A-㉡, B-㉣, C-㉠, D-㉢, E-㉤**
7. 편리성을 중시하는 사회 분위기에서 **이미 편리한 신용카드가 존재**해 주목받지 못했습니다. 또한 암호 기술(은닉 서명)을 썼지만 **중앙기관이 발행하고 이중 지불을 검증**하는 구조였습니다.
8. **① 제3 신뢰 기관 ② 암호학적 증명 ③ CPU 연산 ④ 에너지 소비**

2. 블록체인 기초 (9~17)

9. **A-㉢, B-㉠, C-㉡**
10. **송신자**(트랜잭션을 만든 노드), **수신자**(트랜잭션 대상: 계정, 스마트 계약 등), **값**(전송 화폐의 양 또는 함수 인자). 예: 화폐 전송(잔액 이동), 스마트 계약 실행(함수 호출), 블록 생성
11. **O**
12. 발행 기관이 장부에 Id 2334가 이미 기록되어 있는지 확인합니다. 이미 제출된 번호이므로 **검증 실패(이중 지불)** 로 판정하고 **승인 거절**을 통보합니다. 처음 제출된 번호였다면 장부에 기록하고 검증 성공을 통보합니다.
13. **① 특정 값을 찾는 연산 작업(Mining) ② 한정된 총발행량(2,100만 개) ③ 총발행량과 발행 시간으로 예측 가능**
14. **① 허가형(Permissioned) ② 비허가형(Permissionless) ③ Public(개방형)**
15. **②** — ① 폐쇄형(Private) ③ 컨소시엄 ④ 개방형(Public)
16. **A-㉢, B-㉠, C-㉣, D-㉡**
17. **(나) → (마) → (라) → (가) → (다)**

3. 블록체인을 위한 암호기술 (18~29)

18. **160비트**. 충돌 쌍을 찾는 공격이 가능해졌기 때문입니다(과거 SSL에 쓰였음).
19. **A-㉡, B-㉢, C-㉠**
20. 해시 값은 데이터의 지문 역할을 합니다. 내려받은 파일의 해시를 직접 계산해 **신뢰 사이트가 제공한 해시 값과 비교**하면 파일이 원본 그대로인지(위·변조 여부) 확인할 수 있습니다.
21. **A-㉢, B-㉠, C-㉡, D-㉣**
22. **① 51 ② 14 ③ 5 ④ Keccak**
23. **{해시4, 해시12, 루트해시}** — 해시3과 해시4로 해시34를 만들고, 해시12와 합쳐 루트를 계산한 뒤 루트해시와 비교합니다.
24. **10개** — $\log_2 1024 = 10$, 머클 경로 크기는 O(log N)입니다.

25. ① 받은 메시지 m에 같은 해시 함수를 적용해 h(m)을 계산합니다. ② 서명자의 **공개키**로 서명 s를 검증해 h(m)과 일치하는지 확인합니다. ③ 일치하면 무결성과 서명자 인증이 확인됩니다.

26. d는 **개인키**, n은 **큰 두 소수의 곱(공개 값)** 입니다. 소인수분해되면 안전성이 깨집니다. RSA는 **서명 크기가 크고 생성·검증 속도가 느려** 블록체인에 적합하지 않습니다.

27. **X** — EdDSA는 난수 대신 **메시지 기반 해시 값**을 사용하고 연산 과정이 일정해 부채널 공격에 **강합니다**.

28. ① **불가능** — 최소 2명(t=2)의 서명이 필요하므로 키 1개로는 위조할 수 없습니다. ② **가능** — 남은 2개 키로 2-of-3 조건을 충족합니다.

29. $(1/2)^{10}$ = **1/1024**. 거짓인 문장으로 검증자를 계속 속일 수 없다는 **건전성(Soundness)** 과 관련됩니다.

4. 합의 알고리즘 (30~41)

30. **(나) → (다) → (가)**

31. **A-㉡, B-㉠, C-㉣, D-㉢**

32. **네트워크 지연, 블록 동시 제안, 악의적 공격, 경제적 동기**

33. **X** — 최장 체인 규칙은 **확률적 최종성**을 제공합니다. 즉시·결정적 최종성은 PBFT(2/3 이상 투표)의 특징입니다.

34. **A → B → D** 체인이 선택됩니다(더 길다 = 누적 작업량이 많다). 선택되지 못한 블록 C는 **스테일 블록(Stale Block)** 입니다.

35. **X** — SegWit은 하위 호환되는 **소프트포크**이고 하나의 체인으로 유지되었습니다. 비트코인/비트코인 캐시 분리는 하드포크 사례입니다.

36. **(다) → (마) → (가) → (라) → (나)**

37. **ASIC**(Application-Specific Integrated Circuit). 특정 하드웨어가 과도하게 유리하지 않도록 퍼즐을 설계하는 것도 중요하다는 점을 보여 줍니다.

38. **슬래싱(Slashing)**

39. 블록 승인 시 **대표자와 그 대표자에게 투표한 노드 모두**에게 보상이 분배됩니다. 장점: **매우 높은 확장성, 효율적인 거버넌스**(문제 시 대표자 교체 가능). 단점: **소수 대표자에게 권력 집중**(탈중앙화 약화), 투표 참여 저조.

40. ① 배신자 C가 메시지를 바꿔 전달해 **A, B만 6시에 공격 → 패배**합니다. ② A, B, D가 각각 '6시 2개, 2시 1개'를 받아 2/3 다수결로 **6시 공격을 결정 → 승리**합니다. 일반화하면 f개의 악의적 노드를 허용하려면 **3f+1개** 노드가 필요합니다.

41. ③ — PBFT는 참여자가 늘면 메시지가 폭증해 성능이 떨어지므로 **허가형** 환경에 적합합니다.

5. 비트코인 (42~55)

42. **UTXO 기반 트랜잭션, 채굴 경쟁을 통한 PoW, Longest Chain 선택 규칙, 보상 규칙(코인베이스 + 수수료)**

43. 비트코인 주소는 **화폐의 잠금·해제 조건 라벨**입니다. 이더리움 주소는 **계정/컨트랙트의 고정 식별자**입니다. 둘 다 노드의 IP 주소와는 다른 개념입니다.

44. **A-㉢, B-㉣, C-㉠, D-㉡**

45. ④ — Reference Client(Bitcoin Core)에 대한 설명입니다. SPV 노드는 스마트 기기처럼 송금 기능만 필요한 소형 장비에 적합합니다.

46. **디지털 서명**(공개키로 검증되는지), **UTXO**(이미 사용된 적 없는지), **스크립트**(잠금 해제 조건 충족). 저장 위치: **메모리풀(MemPool)**

47. **이전 블록 해시**(자신의 최신 블록 해시와 일치), **트랜잭션**(모두 유효한지 재검증), **머클 루트**(바디의 거래 내역으로 검증되는지), **작업 증명 결과**(넌스와 해시 값이 난이도 기준 충족)

48. ① **공개키** ② **개인키** ③ **P2SH**

49. 입력 3.5 BTC → 출력: **B에게 3 BTC**, **A에게(거스름) 0.45 BTC**. 차액 0.05 BTC가 수수료입니다. 사용한 두 UTXO는 파기되고, A의 잔액은 남은 UTXO의 합 0.7 + 0.45 = **1.15 BTC**입니다.

50. 3.125 + 0.42 = **3.545 BTC** — 블록 보조금 + 블록 내 모든 수수료 합

51. 450,000 ÷ 210,000 = 2.14… → 반감기 **2회** 경과 → 50 → 25 → **12.5 BTC**

52. ① E = 0x17 = **23**, C = **0x034219** ② C 뒤의 0: E − 3 = **20바이트**, C 앞의 0: 32 − 3 − 20 = **9바이트**
T = 0x000000000000000000**034219**0000…(20바이트)

53. **X** — C = 0x80ffff의 최상위 비트가 1입니다. 최상위 1비트는 부호를 나타내며 **0이어야 유효**합니다.

54. 2,016 × 10분 = 20,160분 = **14일(약 2주)**

55. (a) **X** — 트랜잭션 카운터를 제외한 80바이트 헤더로 해시합니다. (b) **O** (c) **O** (d) **X** — 채굴자는 수수료 등을 고려해 **자율적으로** 선택하며, 수수료가 높은 트랜잭션이 우선됩니다.

6. 이더리움 기초 (56~64)

56. ① **2015년 7월 30일** ② **2022년 9월 15일**(The Merge)

57. **X** — 오보로 밝혀졌습니다. 다만 공격 가능성 때문에 시장 불안이 커져 당일 11% 급락했습니다.

58. ① **Ghash.io** ② **Foundry USA와 AntPool**

59. **③** — 이전 출력을 참조하는 것은 UTXO 모델의 특징입니다. 강의의 계정 모델 장점은 ①②④와 '거래 기록 생성이 비교적 쉬움'입니다.

60. 동전을 넣고 버튼을 누르면 커피가 나오듯, **입력 조건이 충족되면 자동 실행되고 미충족되면 실행되지 않습니다.** 판매자와 구매자가 제3자 없이 계약을 스스로 집행할 수 있습니다. '결정적'이란 **같은 입력(트랜잭션·메시지 호출)에 항상 같은 결과**를 낸다는 뜻입니다.

61. **(나) → (라) → (가) → (다)**

62. ① **튜링-완전** ② **계정 모델** ③ **UTXO 모델** ④ **12** ⑤ **10** ⑥ **Gas Limit**(30M-60M Gas)

63. ① 60,000 × 15 = **900,000 gwei = 0.0009 ETH** ② (100,000 − 60,000) × 15 = **600,000 gwei** 환불. 무한 루프라면 가스를 모두 소진하므로 100,000 × 15 = **1,500,000 gwei(0.0015 ETH)** 를 내고 환불은 없으며, 상태 변경은 취소(reverted)됩니다.

64. **O**

7. 이더리움 (65~75)

65. CA에는 **개인키가 없어** 트랜잭션에 서명할 수 없습니다. 다른 CA나 EOA로부터 받은 트랜잭션(메시지)에 **응답**하는 방식으로 동작하며, 메시지가 조건을 충족하면 코드가 실행됩니다. 그래서 모든 트랜잭션의 출발점은 항상 EOA입니다.

66. ① EOA: **계정에서 전송된 트랜잭션 개수**(트랜잭션이 한 번만 처리되도록 보장) ② CA: **해당 계정이 생성한 계약의 수**

67. **X** — codeHash는 다른 필드와 달리 **변경 불가**입니다. EOA가 공백 문자의 해시를 저장한다는 부분은 맞습니다.

68. ① 0.25 × 10^{18} = **2.5×10^{17} Wei** ② **10^{9} Wei**

69. ① **160비트** ② **40글자**(1바이트 = 16진수 2글자) ③ **EIP-55 checksum**

70. **③** — ① 풀 노드는 과거 상태를 삭제(프루닝)합니다. ② 라이트 노드는 최소 데이터만 저장하고 풀 노드에 의존합니다.

71. **A-㉣, B-㉡, C-㉠, D-㉤, E-㉢**

72. 실제 가스 단가 = 20 + 3 = 23 gwei
① 50,000 × 23 = **1,150,000 gwei** (0.00115 ETH)
② 50,000 × 20 = **1,000,000 gwei** 소각
③ 50,000 × 3 = **150,000 gwei**
④ (30 − 23) × 50,000 = **350,000 gwei**

73. V **1바이트**, R **32바이트**, S **32바이트** → 합계 **65바이트**. CA 생성 시 `value`는 새 계약 계정의 **첫 잔액**이 됩니다(`to`는 null).

74. ① 32 × 12 = **384초(약 6분)** ② ㉠ **LMD-GHOST** ㉡ **Casper FFG** ㉢ **2/3**

75. **(나) PoS 전환(2022) → (라) L2 활성(2023) → (다) 샤딩/블롭 도입(2024) → (가) 블롭 용량 확대(2025)**

8. DApp 개발환경 구축 (76~83)

76. (A) **Remix IDE, Hardhat** (B) **Geth, Infura** (C) **Ganache, Sepolia** (D) **MetaMask, ethers.js**

77. 공통점: 스마트 계약의 컴파일·배포·테스트 등 **전 개발 과정**을 제공합니다(Hardhat도 로컬 블록체인 제공). Hardhat은 Node.js 기반 CLI로, **팀 개발 환경을 프로젝트 단위로 공유**하거나 **여러 계약 배포를 자동화**하고 계약·계정 간 상호작용을 반복 검증할 때 유리합니다.

78. **X** — **2023년에 업데이트 지원이 중단**되었습니다.

79. **클라우드 기반 RPC 노드 서비스**를 제공합니다. **별도 노드를 설치하지 않고도** 블록체인과 연결할 수 있습니다.

80. **X** — 단순한 DApp은 DB 없이 브라우저에서 ethers.js와 지갑으로 스마트 계약과 직접 통신할 수 있습니다. Node.js는 DB 저장, 세션 관리, 이벤트 수집 같은 오프체인 기능이 필요할 때 씁니다.

81. **A-㉣, B-㉡, C-㉠, D-㉢**

82. ① **Wei** ② **[Browser extension] → [Sepolia Testnet - MetaMask]**

83. (a) **O** (b) **X** — 가스 수수료가 **발생**하며, 그래서 Faucet으로 테스트용 ETH를 받습니다. (c) **O** (d) **O**

9. 솔리디티 — 8주차 (84~100)

84. **X** — 솔리디티는 **정적 타입** 언어로 **컴파일 시** 타입을 검사하며, 구문은 **C에 가깝습니다**. 상속·인터페이스·라이브러리 같은 객체지향 요소를 포함합니다.

85. 거의 대부분의 스마트 계약이 솔리디티로 구현되는 현상입니다. 장점: **상호운용성, 조합성, 개발 비용 절감**. 단점: **공통 취약점이 생태계 전체에 내재**하고, 고수준 언어라 비효율적입니다.

86. **A-㉢, B-㉣, C-㉠, D-㉡**

87. (a) **0.8.3, 0.8.25** — ^0.8.3 = 0.8.3 이상 0.9.0 미만
(b) **0.8.25, 0.9.0** — 0.8.3 초과
(c) **0.8.3** — `=0.8.3`과 같은 의미
(d) **0.8.2, 0.8.3**

88. ① **정확히 0.6.8만** ② 솔리디티 0.6 이상에서는 SPDX 라이선스 식별자가 없으면 **경고**가 납니다(오류는 아님). ③ 컨트랙트가 **배포될 때 한 번만** 최초 실행되는 초기화 함수입니다. ④ `greet`가 **private**이라 getter가 자동 생성되지 않기 때문입니다. `public`이었다면 `greet()` getter가 자동으로 만들어집니다.

89. **A-㉡, B-㉠, C-㉣, D-㉢**

90. `msg.sender` = **컨트랙트 A의 주소**(B를 직접 호출한 주소), `tx.origin` = **Alice의 주소**(트랜잭션을 시작한 원래 주소)

91. ① **컨트랙트 내부, 함수 외부** ② **storageRoot** ③ **스택이나 메모리** ④ **함수 실행이 끝나면 사라짐** ⑤ **저렴**

92. 상태 변수: **str** / 지역 변수: **num1, num2, addr** / 전역 변수: **block.timestamp, msg.sender**
num2와 addr는 전역 변수의 값을 담는 **지역 변수**입니다.

93. 값 타입: **bool, uint, address, enum, bytes32**(고정 크기 바이트 배열) / 참조 타입: **string, 배열, struct, mapping, bytes**(동적 크기 바이트 배열)

94. ① **참조** ② **복사** ③ **복사** ④ **참조** ⑤ **복사** — 같은 영역끼리는 참조(얕은 복사), 다른 영역 사이는 복사입니다. calldata → storage도 복사입니다.

95. ① int8: **–128 ~ 127**($-2^7 \sim 2^7-1$), uint8: **0 ~ 255**($0 \sim 2^8-1$) ② **uint256**(int는 int256) ③ `**`

96. getTwo → **2**: 변수 a/b는 uint이므로 3/2 = 1(소수점 버림), 1 × 2 = 2
getThree → **3**: 리터럴끼리의 상수 식은 컴파일 시 정확한 값으로 계산되어 3/2 × 2 = 3
shift → **[256, 2]**: $16 \times 2^4 = 256$, $16 \div 2^3 = 2$

97. ④ — `delegatecall`은 다른 컨트랙트의 **코드만 빌려 쓰고, 저장소는 현재 컨트랙트 것**을 사용합니다. 참고로 `transfer` / `send`는 0.8.31부터 deprecated입니다.

98. ① `b[0] = 'b'`, `b[1] = 'a'` ② `bytes2`는 고정 크기(bytes1~bytes32) 바이트 배열이라 **값 타입**입니다. `bytes`는 **동적 크기** 바이트 배열이라 참조 타입이므로 저장 위치(`memory`)를 명시해야 합니다.

99. ① **0** (Red) — setColor()를 호출하지 않으면 기본값 0 ② **2** (Green) ③ **1** (Blue)

100. ① **100** ② `this.e`는 주소(현재 컨트랙트 `this`)를 통해 호출하는 **외부 함수 타입**, `g()`는 현재 함수 안에서 참조하는 **내부 함수 타입** ③ 함수의 **4바이트 식별자**(`bytes4`) ④ **불가능** — `internal` 함수는 외부에서 호출할 수 없으므로 Remix에 버튼도 생성되지 않습니다.